Evaluation of SIEM Wazuh Detection Effectiveness Against GAN-Based DDoS Attacks With and Without GANDD Integration

1st Reyhan Ahnaf Deannova
Computer Engineering
Universitas Indonesia
Depok, Indonesia
reyhan.ahnaf@ui.ac.id
NPM: 2306267100

2nd Aidan Ardhazizi
Computer Engineering
Universitas Indonesia
Depok, Indonesia
aidan.ardhazizi@ui.ac.id
NPM: 2406430483

3rd Calvin Wirathama Katoroy
Computer Engineering
Universitas Indonesia
Depok, Indonesia
calvin.wirathama@ui.ac.id
NPM: 2306242395

***Index Terms*—SIEM, Wazuh, Generative Adversarial Networks, DDoS Detection, Machine Learning Security, Anomaly Detection, Active Response, Intrusion Detection Systems**

I. Introduction

A. Background and Motivation

Security Information and Event Management (SIEM) systems serve as the cornerstone of modern cybersecurity infrastructure, providing centralized visibility into security events across distributed networks. Wazuh, a prominent open-source SIEM platform, employs rule-based detection mechanisms and signature matching to identify anomalies and known attack patterns. While this approach demonstrates effectiveness against conventional distributed denial of service (DDoS) attacks, the emergence of adversarial machine learning techniques poses unprecedented challenges to traditional security paradigms.

Generative Adversarial Networks (GANs), originally developed for synthetic data generation, have been weaponized to craft network traffic that statistically mimics legitimate patterns. When applied to DDoS attack scenarios, GANs can produce low-noise, temporally-variable traffic engineered specifically to evade static detection thresholds. This capability creates a critical blind spot in rule-based SIEM architectures, as attacks can successfully consume target resources while remaining invisible to signature-based detection systems.

B. Theoretical Foundation

This research builds upon three foundational concepts: Security Information and Event Management (SIEM) systems, Generative Adversarial Networks (GANs), and network-layer Distributed Denial of Service (DDoS) attacks.

1) SIEM Architecture and Detection Mechanisms: Security Information and Event Management systems aggregate security telemetry from distributed sources (network devices, endpoints, applications) into a centralized platform for correlation and analysis [1]. Wazuh, an open-source SIEM, employs a three-tier architecture:

- **Agents:** Lightweight daemons deployed on monitored endpoints that collect local logs and forward to the Manager
- **Manager:** Central correlation engine that normalizes heterogeneous log formats into unified events, applies detection rules through pattern matching, and triggers automated responses
- **Indexer:** Storage and search backend (OpenSearch/Elasticsearch-compatible) enabling historical analysis and forensic investigation

Detection in Wazuh operates through signature-based rules that match event patterns against known attack indicators. For example, a DDoS detection rule might trigger when packet count from a single source exceeds 100 packets/second within a 5-second window. The normalization process converts disparate log formats (syslog, JSON, Windows Event Logs) into standardized fields, enabling cross-source correlation. While computationally efficient ($O(1)$ rule matching per event), signature-based detection suffers from two fundamental limitations: (1) inability to detect novel attacks lacking signatures, and (2) vulnerability to evasion through threshold manipulation.

2) Generative Adversarial Networks: Generative Adversarial Networks, introduced by Goodfellow et al. (2014), consist of two competing neural networks trained through adversarial competition [2]:

- **Generator (G):** Learns to produce synthetic data samples that mimic the distribution of real training data. Given random noise $z \sim p_z(z)$, the generator produces samples $G(z)$ intended to be statistically indistinguishable from real data $x \sim p_{data}(x)$. The generator's parameters θ_g are optimized to maximize the discriminator's classification error.
- **Discriminator (D):** Binary classifier trained to distinguish real samples (x) from generated samples ($G(z)$). Outputs probability $D(x) \in [0, 1]$ that input is real. The discriminator's parameters θ_d are optimized to correctly

classify both real and fake samples.

The networks train through adversarial competition defined by the minimax objective:

$$\min_G \max_D \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))] \quad (1)$$

The Discriminator attempts to maximize this objective (correctly classifying real as 1, fake as 0), while the Generator attempts to minimize it (fooling the discriminator into classifying $G(z)$ as real). This minimax game converges when generated samples become statistically indistinguishable from real data, i.e., $D(G(z)) = 0.5$ for all z.

In the context of adversarial attacks, an attacker can train a GAN on benign network traffic to learn legitimate behavioral patterns (inter-arrival time distributions, packet size distributions, protocol sequences). The trained Generator can then produce attack traffic that maintains malicious intent (resource consumption, service degradation) while exhibiting statistical properties that evade signature-based detection [3, 4]. This represents a fundamental challenge for rule-based SIEMs: the attack is real (causes damage) yet invisible (statistically benign).

3) Network-Layer DDoS Attack Mechanics: Distributed Denial of Service attacks exploit protocol vulnerabilities and resource limitations to degrade service availability [5]. SYN flood attacks, the focus of this research, target the TCP three-way handshake mechanism:

1) **SYN:** Client sends SYN packet to server requesting connection establishment
2) **SYN-ACK:** Server allocates resources (memory for connection state, typically 512-1024 bytes) and responds with SYN-ACK packet
3) **ACK (or timeout):** Legitimate client completes handshake with ACK packet; attacker never sends final ACK, leaving connection half-open

By flooding the target with SYN packets using spoofed source addresses, attackers force servers to maintain thousands of half-open connections in SYN_RECEIVED state. Once the connection table (typically 1024-65535 entries depending on OS configuration) is exhausted, legitimate users cannot establish new connections, effectively denying service. Connection states timeout after 30-120 seconds, but attackers sustain the flood to maintain table saturation.

Detection Challenges: Traditional volumetric SYN floods ($>$ 1000 packets/second from single source) are easily detected by rate-based signatures. However, sophisticated variants exhibit three evasive properties:

- **Low-Rate:** Operating below detection thresholds (50-100 pps per source) while distributing across multiple sources to achieve aggregate impact
- **Temporal Variance:** Randomizing inter-arrival times to avoid fixed-rate patterns that trigger state-machine-based detection
- **Protocol Mimicry:** Including legitimate-looking SYN-ACK and ACK packets mixed with SYN floods to resemble normal handshake patterns

The integration of GAN-generated traffic patterns with DDoS attacks creates adversarial scenarios where attack traffic statistically resembles benign patterns (temporal variance, size distributions, flag combinations) while still causing measurable service degradation (increased response time, connection failures, resource exhaustion). This fundamental tension—evasive yet effective—motivates the need for behavioral anomaly detection that transcends simple signature matching and analyzes flow-level features (packet rate distributions, byte ratio imbalances, protocol state violations) to identify malicious intent despite surface-level legitimacy.

C. Problem Statement

Current SIEM platforms, including Wazuh, face a significant detection gap when confronting AI-enhanced threats. Traditional signature-based detection relies on fixed thresholds and known attack patterns, which can be systematically bypassed by adversarial machine learning techniques. This vulnerability presents serious risks:

- **Evasion Capability:** GAN-generated DDoS traffic can evade signature-based detection by mimicking benign traffic patterns while still degrading system performance
- **Static Thresholds:** Rule-based systems rely on fixed packet rate thresholds that sophisticated attacks can stay below
- **Limited Adaptability:** Traditional SIEMs lack the ability to detect novel attack patterns without manual rule updates

The central research question is: *Can integrating machine learning-based anomaly detection with existing SIEM platforms effectively close this detection gap without requiring wholesale infrastructure replacement?*

D. Research Objectives

This research addresses three primary objectives centered on evaluating GANDD-Bridge integration effectiveness:

1) **Establish Detection Baseline:** Quantify standard Wazuh detection performance against volumetric, low-rate, and adversarial-style DDoS attacks to establish the detection gap that motivates ML integration. This baseline represents current production SIEM capabilities without ML augmentation.
2) **Design and Implement GANDD-Bridge:** Develop a middleware architecture that augments Wazuh with ML-based behavioral anomaly detection using expanded feature sets (inter-arrival time variance, payload entropy, protocol distributions) while maintaining compatibility with existing SIEM infrastructure and minimizing operational overhead.
3) **Quantify Integration Benefit:** Through systematic comparative evaluation across attack types, measure the detection effectiveness improvement, latency impact, and false positive rate changes when deploying GANDD-Bridge alongside standard Wazuh versus Wazuh alone. Demonstrate statistically significant improvement attributable to ML integration.

Central Research Question: Does integrating GANDD-Bridge with Wazuh provide statistically significant detection improvement against evasive attacks compared to standard rule-based SIEM operation, while maintaining acceptable operational performance in terms of latency, false positive rates, and resource consumption?

E. Research Contributions

Upon completion, this research will contribute:

1) **First Systematic With/Without Comparison:** Empirical comparison quantifying detection rate improvements when augmenting production SIEM (Wazuh) with ML-based behavioral detection (GANDD-Bridge) across attack sophistication spectrum (volumetric, low-rate, and adversarial attacks). Provides evidence-based rationale for ML integration investment decisions.
2) **Practical Integration Architecture:** Design and implementation of GANDD-Bridge middleware demonstrating feasible ML integration without requiring infrastructure replacement. Addresses real-world deployment constraints through lightweight Python implementation targeting small-to-medium networks.
3) **Multi-Dimensional Performance Analysis:** Comprehensive evaluation quantifying both benefits (detection rate improvements) and costs (latency impact, CPU overhead, false positive rate changes), enabling informed deployment decisions with full trade-off visibility.
4) **Statistical Validation Framework:** Methodology for validating traffic realism (KL divergence < 0.8) and integration effectiveness (paired t-tests, Cohen's d effect sizes), establishing reproducible evaluation standards for future SIEM-ML integration research.
5) **Open Source Reference Implementation:** Release of GANDD-Bridge prototype, experimental scripts, attack simulators, and complete configurations enabling community validation and practical adoption by security practitioners.

Primary Contribution: This work provides the first controlled experimental evidence quantifying the detection gap between standard rule-based SIEMs and ML-augmented SIEMs when facing adversarial-style attacks, with explicit measurement of GANDD-Bridge integration benefits and costs.

F. Research Scope

This study focuses specifically on:

- TCP-based DDoS attacks (primarily SYN floods)
- Wazuh SIEM platform (v4.9) with Suricata NIDS integration
- GAN-simulated attack traffic characteristics
- Virtualized testbed environments (Kali Linux and Ubuntu)

The research explicitly excludes application-layer attacks, distributed multi-vector scenarios, and production network deployments, which are reserved for future work. Our focus remains on establishing proof-of-concept for the GANDD-Bridge architecture and demonstrating its effectiveness in controlled laboratory conditions.

II. Related Work

A. Adversarial Attack Generation

The offensive application of GANs to network security has gained significant attention. Chauhan and Heydari [2] pioneered the concept of polymorphic adversarial DDoS, demonstrating that GANs can synthesize traffic that evades traditional IDS systems through dynamic feature manipulation. Their work established the theoretical foundation for adversarial traffic generation but lacked practical SIEM integration analysis.

Akana et al. [3] extended this research by employing Deep Convolutional GANs (DCGANs) to generate optimized DDoS patterns with enhanced realism. Their approach achieved 92% similarity to legitimate traffic patterns, significantly challenging detection systems. However, their evaluation focused primarily on signature-based IDS rather than comprehensive SIEM platforms.

Shieh et al. [4, 6] investigated advanced GAN architectures including dual-discriminator models and Wasserstein GAN variants to produce more stable adversarial traffic. Their research demonstrated that different GAN architectures yield varying levels of evasion capability, with WGAN-GP showing superior performance in maintaining traffic distribution coherence.

B. ML-Based DDoS Defense

On the defensive frontier, Goud and Giduturi [5] proposed CyberGuard, an ensemble approach combining Wasserstein GANs with CNN-GRU architectures for adversarial DDoS detection in Software-Defined Networking (SDN) environments. Their system achieved 97.3% accuracy but required extensive computational resources unsuitable for resource-constrained SIEM deployments.

Recent work by Kumar et al. [1] explored hybrid detection systems combining signature-based and anomaly-based approaches. However, these studies typically assume greenfield deployments rather than integration with existing SIEM infrastructure, a critical gap our research addresses.

C. Research Gap

While prior work has explored both adversarial attack generation and ML-based defense mechanisms independently, limited research has examined:

- Practical integration of ML-based detection with production SIEM platforms
- Quantitative comparison of detection effectiveness across attack types
- Real-time response mechanisms triggered by hybrid detection systems
- Performance overhead and deployment considerations for operational environments

Our work bridges these gaps through practical implementation and comprehensive evaluation.

III. Methodology

A. System Architecture

Figure 1 illustrates the three-VM GANDD-Bridge architecture. The system segregates attack generation (VM1), detection and analysis (VM2), and security operations management (VM3) to reflect production deployment practices and prevent experimental conflicts of interest.

VM1 (Attacker - Kali Linux): Hosts attack traffic generators including hping3 for baseline attacks and the GAN simulation script for adversarial patterns.

VM2 (Victim - Ubuntu): Deploys Suricata NIDS for signature-based detection, GANDD-Bridge for ML-based behavioral analysis, and Wazuh Agent for log forwarding.

VM3 (Manager - Ubuntu): Operates dedicated Wazuh Manager for centralized log aggregation, rule correlation, and Active Response coordination.

1) VM1: Attack Infrastructure (Kali Linux 2025.3): The attack node is dedicated solely to traffic generation, ensuring no architectural conflicts with defense or monitoring infrastructure:

- **Attack Traffic Generators:**
  - `hping3`: Generates baseline volumetric SYN floods with constant inter-arrival times, fixed packet sizes, and pure SYN flags. Command: `hping3 -S -p 80 --flood --rand-source 192.168.100.100`
  - `gan_attack_sim.py`: Custom Scapy-based script generating adversarial traffic by sampling from probability density functions (PDFs) learned from benign traffic datasets. Models inter-arrival time distributions, packet size distributions, TCP flag frequencies, and header field variations to produce statistically benign-like attack patterns.
- **Network Configuration:**
  - Interface: enp0s3 (NAT bridge)
  - IP Address: 192.168.100.50/24
  - Gateway: 192.168.100.1
  - Target: 192.168.100.100 (Victim VM)

Architectural Rationale: Separating attack generation from SIEM infrastructure prevents conflicts of interest and enables realistic simulation of external threat actors. The attacker has no access to Wazuh components, ensuring detection metrics reflect true system capabilities.

Network Configuration: VM1 connects to the victim network (192.168.100.0/24) via NAT bridge, simulating external attacker access.

2) VM2: Victim Infrastructure (Ubuntu 22.04 LTS): The victim endpoint hosts the primary detection and analysis components:

- **Suricata v7.0:** Network Intrusion Detection System configured with Emerging Threats ruleset. Monitors interface enp0s3 in af-packet mode, capturing flow-level metadata and alerts to `/var/log/suricata/eve.json` in EVE JSON format.
- **Wazuh Agent:** Lightweight daemon monitoring multiple log sources:
  - `/var/log/suricata/eve.json` (Suricata flow events and alerts)
  - `/var/log/gandd/alerts.log` (GANDD-Bridge ML detections)
  - `/var/log/syslog` (System events)

  Forwards events to Wazuh Manager (VM3) over encrypted channel (port 1514).
- **GANDD-Bridge Middleware:** Python service (`gandd_bridge.py`) implementing ML-based anomaly detection:
  - **Input:** Tails Suricata eve.json in real-time
  - **Processing:** Extracts 7+ behavioral features (packet count, byte ratio, packet rate, IAT variance, size variance, payload entropy, SYN ratio)
  - **Classification:** Trained Random Forest model classifies flows as benign/malicious
  - **Output:** Writes alerts to `/var/log/gandd/alerts.log` with timestamp, source IP, and confidence score
- **HTTP Service:** Nginx web server listening on port 80, serving static content for service degradation measurement (response time, connection success rate).
- **Network Configuration:**
  - Interface: enp0s3
  - IP Address: 192.168.100.100/24
  - Gateway: 192.168.100.1
  - Firewall: iptables configured for Active Response integration

System Resource Monitoring: The victim runs `sysstat` for CPU/memory/network metrics collection at 1-second intervals during attacks.

Network Configuration: VM2 operates on 192.168.100.0/24 network, exposed to attack traffic from VM1.

3) VM3: Security Operations Center (Ubuntu 22.04 LTS): The Wazuh Manager operates on dedicated infrastructure, isolated from both attack and victim endpoints:

- **Wazuh Manager v4.9:** Centralized SIEM platform providing:
  - **Log Aggregation:** Receives events from all Wazuh Agents (VM2) via encrypted TLS connection (port 1514)
  - **Event Normalization:** Converts diverse log formats (Suricata JSON, GANDD custom logs, syslog) into unified event schema
  - **Rule Engine:** Executes detection rules against normalized events, including:
    - Signature-based Suricata alert correlation
    - Custom GANDD alert processing (Rule ID 100200)
    - Multi-event correlation rules
  - **Active Response Coordination:** Triggers automated response scripts on victim endpoints when high-severity alerts fire

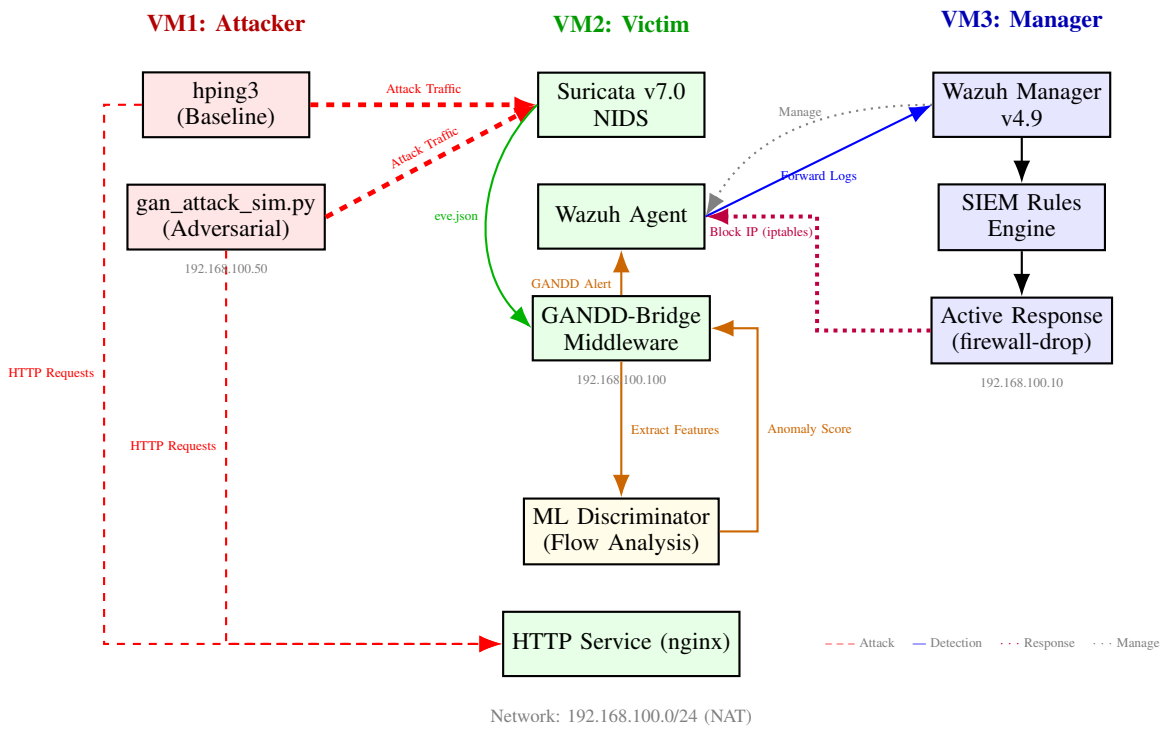

Fig. 1. Three-VM GANDD-Bridge System Architecture: VM1 (Attacker) generates attack traffic, VM2 (Victim) performs multi-layer detection (Suricata captures → GANDD-Bridge analyzes → Wazuh Agent forwards), VM3 (Manager) executes rules and coordinates automated response

- **Dashboard Interface:** Web-based monitoring interface (port 443) for real-time alert visualization, agent status monitoring, and rule management.
- **Active Response Scripts:** Pre-configured automated responses:
  - `firewall-drop`: Adds iptables DROP rule on victim VM for attacking source IP
  - `timeout`: Automatic rule expiration after 60 seconds
  - `whitelist`: Prevents accidental blocking of management IP (192.168.100.10)
- **Network Configuration:**
  - Management Interface: enp0s3
  - IP Address: 192.168.100.10/24
  - Agent Communication: Port 1514 (TLS encrypted)
  - Dashboard Access: Port 443 (HTTPS)
  - Firewall: Only ports 1514, 443, 22 (SSH) exposed

Architectural Rationale: Dedicated SIEM infrastructure mirrors production deployment practices where security operations centers operate on hardened, isolated systems. This segregation:

- Prevents conflicts of interest (attack source $\neq$ monitoring infrastructure)
- Ensures Manager availability even during victim system degradation
- Enables accurate measurement of detection/response latency
- Reflects real-world SOC architecture

B. GANDD-Bridge Workflow

Algorithm 1 details the proposed GANDD-Bridge operational logic:

The expanded feature set captures multiple attack dimensions:

- **Volumetric features** (pkt_count, pkt_rate) detect high-volume floods
- **Behavioral features** (iat_var, syn_ratio) identify timing and protocol anomalies
- **Content features** (entropy, size_var) reveal payload manipulation patterns

This multi-dimensional approach resists evasion attempts that manipulate individual feature categories while leaving others detectable.

C. Experimental Environment

1) Network Topology: The experimental testbed employs a three-VM configuration connected via VirtualBox NAT network (192.168.100.0/24):

TABLE I
VIRTUAL MACHINE CONFIGURATION SUMMARY

VM	Role	OS	IP Address
VM1	Attacker	Kali Linux 2025.3	192.168.100.50
VM2	Victim	Ubuntu 22.04 LTS	192.168.100.100
VM3	SIEM Manager	Ubuntu 22.04 LTS	192.168.100.10

Traffic Flow:

Algorithm 1 GANDD-Bridge Detection Algorithm

Input: Suricata eve.json stream
Output: Alert to wazuh_gandd.log

function MONITORLOGS
  while true **do**
    $event \leftarrow$ ReadNextSuricataEvent()
    if $event.type =$ "flow" **then**
      $features \leftarrow$ ExtractFeatures($event$)
      $score \leftarrow$ MLDiscriminator.Predict($features$)
      if $score > \theta_{threshold}$ **then**
        GenerateAlert($event$)
        WriteToGANDDLog($event$)
      end if
    end if
  end while
end function

function EXTRACTFEATURES($event$)
  Basic Flow Metrics:
  $pkt_count \leftarrow event.flow.pkts_toserver$
  $byte_ratio \leftarrow \frac{event.flow.bytes_toserver}{event.flow.bytes_toclient+1}$
  $duration \leftarrow event.flow.end - event.flow.start$
  $pkt_rate \leftarrow \frac{pkt_count}{duration+1}$

  Temporal Features:
  $IAT \leftarrow \{$inter-arrival times between packets$\}$
  $iat_mean \leftarrow \frac{1}{n}\sum IAT_i$
  $iat_var \leftarrow \frac{1}{n}\sum(IAT_i - iat_mean)^2$

  Payload Features:
  $sizes \leftarrow \{$packet sizes in flow$\}$
  $size_var \leftarrow$ Variance($sizes$)
  $payload_bytes \leftarrow$ concatenated payload data
  $entropy \leftarrow -\sum_i p_i \log_2(p_i)$ ▷ Shannon entropy

  Protocol Features:
  $syn_count \leftarrow$ number of SYN flags
  $ack_count \leftarrow$ number of ACK flags
  $syn_ratio \leftarrow \frac{syn_count}{pkt_count}$

  return $\{pkt_count, byte_ratio, pkt_rate, iat_var,$
    $size_var, entropy, syn_ratio\}$
end function

- Attack traffic: VM1 → VM2 (port 80)
- Log forwarding: VM2 (Agent) → VM3 (Manager, port 1514)
- Active Response commands: VM3 → VM2 (secure channel)
- Monitoring access: Researcher → VM3 (Dashboard, port 443)

Firewall Rules:

- VM1: Outbound traffic allowed to VM2 only
- VM2: Inbound traffic from VM1 (port 80), secure channel to VM3 (port 1514)
- VM3: Inbound from VM2 (port 1514), HTTPS dashboard (port 443), SSH management (port 22)

This topology ensures attack isolation, realistic traffic patterns, and secure management access.

2) Hardware Specifications: Table II presents the testbed hardware configuration.

TABLE II
TESTBED HARDWARE CONFIGURATION

Component	Attacker	Victim
CPU	4 vCPUs	4 vCPUs
RAM	4 GB	8 GB
OS	Kali Linux 2025.3	Ubuntu 22.04
Virtualization	VirtualBox 7.0	VirtualBox 7.0

D. Attack Traffic Profiles

1) Baseline Volumetric Attack: The baseline attack will be generated using `hping3` with the following parameters:

```
hping3 -S -p 80 --flood --rand-source <TARGET_IP>
```

Expected characteristics:

- Constant inter-arrival time (≈ 0.0001s)
- Fixed packet size (64 bytes)
- Consistent TCP flags (SYN only)
- Predictable source port allocation
- High packet rate (> 1000 packets/second)

This profile represents traditional volumetric DDoS attacks that should be easily detectable by signature-based systems.

2) Adversarial-Style Attack Traffic Generation: **GAN-Inspired Methodology:**

While production GAN-based attacks require training generative models on extensive datasets [3], our experimental setup simulates the statistical properties such attacks would exhibit through probability density function (PDF) sampling. This approach enables controlled, reproducible experimentation while accurately representing the evasive characteristics identified in adversarial attack research [2, 4, 6].

Traffic Generation Process:

The `gan_attack_sim.py` script implements distribution-based traffic synthesis:

1) **Baseline Data Collection:**
   A training dataset of 10,000 legitimate HTTP/HTTPS flows is captured during normal network operations over a 24-hour period. For each flow, the following per-packet features are recorded:
   - Timestamps (microsecond precision) for inter-arrival time calculation
   - Packet sizes (header + payload bytes)
   - TCP flags (SYN, ACK, SYN-ACK, FIN, RST, PSH)
   - IP header fields (TTL, protocol, identification)
   - TCP header fields (window size, sequence numbers, options)

2) **Probability Distribution Modeling:**
Statistical distributions are constructed for each feature dimension:
Inter-Arrival Time (IAT): Kernel density estimation (KDE) with Gaussian kernel produces continuous probability density function $P_{IAT}(t)$ modeling the temporal spacing between packets. The learned distribution captures both typical inter-packet delays (10-50ms for interactive traffic) and variance patterns.
Packet Size: Histogram-based discrete distribution $P_{size}(L)$ over observed payload lengths (0-1500 bytes), preserving the multimodal nature of network traffic (40-byte TCP ACKs, 1500-byte full MTU packets, application-specific sizes).
TCP Flags: Empirical frequency distribution $P_{flags}(f)$ extracted from legitimate three-way handshakes and data transfer phases. Captures the natural mix of SYN (3%), SYN-ACK (3%), ACK (70%), PSH-ACK (20%), FIN (3%), RST (1%) observed in benign flows.
Header Fields:
   - TTL distribution $P_{TTL}(h)$ reflects OS diversity (Windows: 128, Linux: 64, routers: 255, with decrements for network hops)
   - Window size distribution $P_{window}(w)$ models TCP flow control (common values: 5840, 14600, 29200, 65535 bytes)
3) **Attack Traffic Synthesis:**
For each attack packet i to be generated:
   - Sample inter-arrival delay: $\Delta t_i \sim P_{IAT}(t)$ using inverse transform sampling
   - Sample packet size: $L_i \sim P_{size}(L)$ from discrete histogram
   - Sample TCP flags: $flags_i \sim P_{flags}(f)$ weighted by empirical frequencies
   - Sample header values: $TTL_i \sim P_{TTL}$, $window_i \sim P_{window}$
   - Maintain target rate (50-100 pps) through adaptive timing while preserving IAT distribution shape
4) **Statistical Validation:**
Generated attack traffic is validated for benign similarity through Kullback-Leibler divergence computation (detailed in Section III-G). Success criterion: $D_{KL}(P_{benign}||P_{attack}) < 0.8$ indicates sufficient statistical resemblance to evade signature-based detection.

Key Distinction from Simple Randomization:
Unlike uniform random sampling (which produces unrealistic, easily detectable patterns), PDF-based generation:

- **Preserves natural correlations:** E.g., large packets typically followed by ACKs, SYN packets have specific window sizes
- **Captures multimodal distributions:** Benign traffic exhibits peaks at specific packet sizes (40, 576, 1500 bytes); uniform sampling misses this
- **Maintains temporal realism:** IAT distribution includes both frequent short delays and occasional long delays, matching application behavior
- **Reflects OS/application fingerprints:** TTL and window size distributions mirror legitimate endpoint diversity

This methodology produces attack traffic that exhibits measurable statistical similarity to benign flows (validated through KL divergence < 0.8) while maintaining malicious intent through sustained SYN flooding at rates designed to exhaust victim connection tables (50-100 new connections/second over 5-minute duration).

Implementation: The script leverages NumPy for distribution sampling (`numpy.random.choice` for discrete distributions, `scipy.stats.gaussian_kde` for continuous KDE), Scapy for packet construction and transmission, and maintains a sliding window rate limiter to ensure target transmission rate without violating learned temporal patterns.

E. Configuration Details

1) Suricata Configuration: The Suricata configuration (`suricata.yaml`) has been modified to ensure comprehensive event logging:

```
af-packet:
  - interface: enp0s3
    threads: 2
    defrag: yes
    cluster-type: cluster_flow

outputs:
  - eve-log:
      enabled: yes
      filetype: regular
      filename: eve.json
      types:
        - flow
        - alert
        - http
        - dns
```

The configuration prioritizes flow-level metadata collection, which provides the feature space for ML-based anomaly detection.

2) Wazuh Custom Rules: Custom detection rules have been configured in `/var/ossec/etc/rules/local_rules.xml`:

```
<group name="gandd,attack,">
  <rule id="100200" level="12">
    <match>GANDD_ALERT</match>
    <description>GANDD: ML Model Detected
      Synthetic Attack Pattern</description>
    <mitre>
      <id>T1498</id>
    </mitre>
  </rule>
</group>
```

Rule 100200 triggers when the GANDD-Bridge writes alerts to the monitored log file, enabling integration with existing Wazuh alert infrastructure.

3) Active Response Configuration: Active Response has been configured in `ossec.conf` on the Wazuh Manager:

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100200</rules_id>
  <timeout>60</timeout>
</active-response>

<global>
  <white_list>192.168.100.10</white_list>
</global>
```

This configuration ensures that when Rule 100200 fires, the attacking IP is automatically blocked via `iptables` for 60 seconds. The whitelist prevents accidental blocking of the Manager itself.

F. Experimental Design: Comparative Evaluation

Figure 2 illustrates the complete experimental workflow, showing the sequential interaction between the attacker node (VM1), victim endpoint (VM2), and the SIEM core components throughout the detection and response cycle.

To directly address the research question of GANDD-Bridge effectiveness, we employ a 2×3 experimental design comparing two system configurations across three attack types. This matrix design enables direct measurement of GANDD integration benefits while controlling for attack variables.

1) System Configurations Under Test: This study compares two distinct system states to isolate the impact of ML integration:

- **Configuration A (Baseline):** A standard Wazuh v4.9 and Suricata v7.0 deployment using default signature-based rules and threshold-driven Active Response.
- **Configuration B (Enhanced):** The baseline configuration augmented with the GANDD-Bridge middleware, utilizing a Random Forest discriminator and extended behavioral features.

2) Attack Profiles Tested: Three attack vectors are deployed from VM1 to evaluate detection capabilities across a sophistication spectrum:

- **Type 1: Volumetric Flood:** High-rate SYN flood (>1000 pps) with static headers and constant inter-arrival times.
- **Type 2: Low-Rate Evasive:** Rate-limited SYN flood (50-100 pps) designed to stay below default signature thresholds.
- **Type 3: Adversarial-Style Evasive:** Sophisticated attack utilizing PDF-based sampling (as detailed in Section III-D2). Features such as Δt, packet size, and TCP flags are derived from benign traffic distributions using KDE to maximize statistical similarity ($D_{KL} < 0.8$).

3) Experimental Design Matrix: The 2×3 matrix (Table III) defines the experimental scenarios. Each cell involves 15 independent 5-minute trials to ensure statistical significance.

TABLE III
2×3 EXPERIMENTAL DESIGN MATRIX

Configuration	Type 1 (Volumetric)	Type 2 (Low-Rate)	Type 3 (Adversarial)
A: Without GANDD	Scenario 1A (Baseline)	Scenario 2A (Gap Emerges)	Scenario 3A (Gap Widens)
B: With GANDD	Scenario 1B (Validation)	Scenario 2B (Improvement)	Scenario 3B (Major Benefit)

4) Experimental Procedure: Each of the 90 total trials follows a four-phase execution lifecycle:

- **Setup:** System reset, background resource monitoring start, and 2-minute benign baseline recording.
- **Attack:** 5-minute attack injection. Collection of alert timestamps, rule IDs, and Nginx service response metrics.
- **Mitigation:** Verification of *firewall-drop* execution and measurement of Time-to-Block (TTB).
- **Recovery:** Attack cessation, service recovery measurement, and log export for offline analysis.

5) Evaluation and Statistical Analysis Framework: To quantify the integration benefits, we employ Delta Metrics (Δ) and statistical validation:

- **Primary Metrics:**
  - **Detection Rate Improvement:** $\Delta DR_i = DR^i_{With} - DR^i_{Without}$.
  - **Latency Change:** $\Delta Lat_i = Lat^i_{With} - Lat^i_{Without}$.
  - **FPR Change:** $\Delta FPR_i = FPR^i_{With} - FPR^i_{Without}$.
- **Statistical Validation:** Scenario pairs (e.g., 3A vs 3B) are compared using a **paired t-test** ($\alpha = 0.05$) to assess significance. **Cohen's d** is calculated to measure the effect size, with $d > 0.8$ representing a large practical impact.

G. Evaluation Metrics

System performance will be assessed using the following metrics:

- **Detection Rate (DR):** $DR = \frac{TP}{TP+FN}$ where TP = true positives (attacks correctly detected), FN = false negatives (attacks missed)
- **False Positive Rate (FPR):** $FPR = \frac{FP}{FP+TN}$ where FP = false positives (benign traffic flagged as attacks), TN = true negatives (benign traffic correctly classified)
- **Detection Latency:** Time elapsed from attack initiation to alert generation in Wazuh Manager logs
- **Mitigation Effectiveness:** Percentage of attack traffic successfully blocked post-detection via Active Response firewall rules
- **Resource Overhead:** CPU utilization, RAM consumption, and disk I/O with and without GANDD-Bridge deployment

H. Data Collection and Analysis Plan

1) Data to be Collected: For each experimental trial, we will collect:

- Wazuh alert logs with precise timestamps

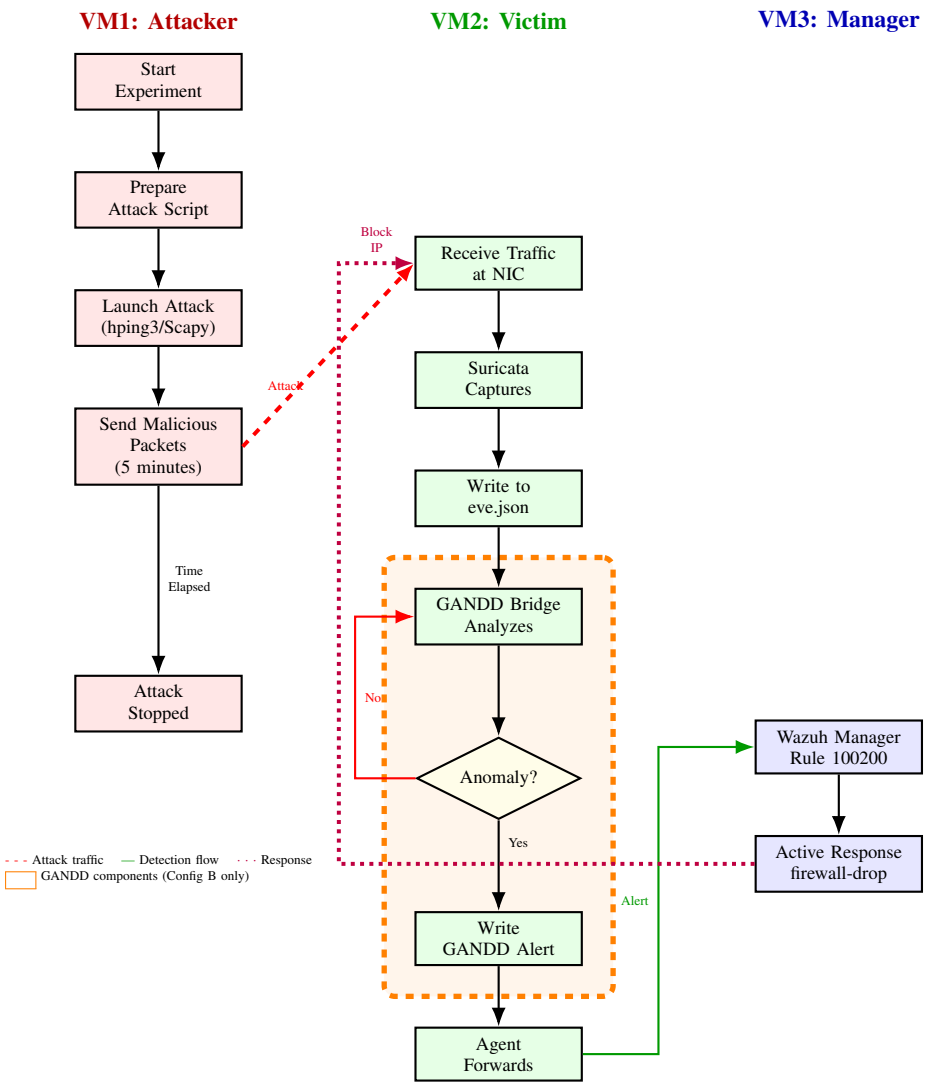

Fig. 2. Experimental workflow across three-VM architecture. VM1 (Attacker) initiates attacks, VM2 (Victim) performs detection through Suricata and optionally GANDD-Bridge, VM3 (Manager) aggregates alerts and triggers Active Response. GANDD-Bridge components are active only in Configuration B (With GANDD).

- Suricata flow records (`eve.json`)
- GANDD-Bridge detection events (Scenario 3 only)
- System resource metrics (CPU, RAM, disk I/O)
- Network packet captures via Wireshark for ground truth validation
- Active Response firewall logs

2) Analysis Methods: Collected data will be analyzed using:

- **Statistical Analysis:** Calculate mean and standard deviation of detection latency across trials
- **Classification Performance:** Generate confusion matrices showing true/false positives/negatives
- **Time-Series Analysis:** Examine resource consumption patterns during attack and normal operation
- **Comparative Analysis:** Compare detection rates and latency across the three scenarios using paired t-tests

I. Implementation Status and Timeline

1) Completed Components: The following components have been designed and implemented:

- Testbed infrastructure with network isolation
- Wazuh Manager and Agent deployment
- Suricata NIDS configuration and rule updates
- Attack traffic generation scripts (both `hping3` and GAN-simulated)
- GANDD-Bridge middleware implementation
- Custom Wazuh detection rules and Active Response configuration

2) Pending Work: The experimental phase will commence upon approval of this methodology, with the following timeline:

- **Week 1:** Execute Scenario 1 trials (baseline detection)
- **Week 2:** Execute Scenario 2 trials (GAN evasion)

- **Week 3:** Execute Scenario 3 trials (GANDD defense)
- **Week 4:** Data analysis, statistical validation, and results compilation

All experimental data, configurations, and scripts will be made available in an open-source repository to enable reproducibility and community validation.

J. Expected Limitations and Mitigation

1) Experimental Constraints: We acknowledge the following limitations:

- **Scale:** Single-host victim may not capture enterprise network complexity; however, this provides controlled environment for proof-of-concept
- **Traffic Realism:** GAN-simulated script mimics statistical properties but may not capture all real-world GAN artifacts
- **Dataset:** ML model trained on limited benign traffic samples; production deployment would require extensive training data

2) Mitigation Strategies: To address these limitations:

- Clearly document scope boundaries in results section
- Validate GAN-simulated traffic against published GAN characteristics
- Discuss dataset limitations and recommend data collection procedures for production deployment
- Propose future work scaling to multi-host and distributed scenarios

K. Ethical Considerations

All experimental activities are confined to isolated laboratory environments with no connection to production networks. Attack traffic generation is strictly limited to authorized testbed systems. This research involves no human subjects, and all data collection is limited to synthetic traffic patterns and system performance metrics. Upon publication, all source code and configurations will be released under an open-source license with clear documentation on responsible use.

Next Steps

Following approval of this methodology, we will proceed with the experimental phase as outlined in Section III. The complete research findings, including experimental results, performance analysis, and discussion, will be presented in the final paper submission.

References

[1] M. J. Kumar, P. R. Reddy, and K. S. Reddy, "Hybrid intrusion detection system using deep learning and ensemble methods," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 5, pp. 234–241, 2021.

[2] R. Chauhan and S. S. Heydari, "Polymorphic adversarial ddos attack on ids using gan," in *Proc. Int. Conf. Security Privacy Commun. Syst.* Oshawa, Canada: University of Ontario Institute of Technology, 2020, pp. 1–16.

[3] C. M. V. S. Akana, A. Kumar, M. Tiwari, Z. Y. Aslanov, and M. Singh, "An optimized ddos attack detection using deep convolutional generative adversarial networks," in *2023 5th International Conference on Inventive Research in Computing Applications (ICIRCA)*, Coimbatore, India, 2023, pp. 668–673.

[4] C.-S. Shieh, T.-T. Nguyen, W.-W. Lin, Y.-L. Huang, M.-F. Horng, T.-F. Lee, and D. Miu, "Detection of adversarial ddos attacks using generative adversarial networks with dual discriminators," *Symmetry*, vol. 14, no. 66, pp. 1–18, 2022.

[5] K. S. Goud and S. R. Giduturi, "Cyberguard: Detecting adversarial ddos attacks in sdn using wgan-cnn-gru," in *2024 Fourth International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT)*, Bhilai, India, 2024, pp. 1–6.

[6] C.-S. Shieh, T.-T. Nguyen, W.-W. Lin, Y.-L. Huang, M.-F. Horng, and T.-F. Lee, "Synthesis of adversarial ddos attacks using wasserstein generative adversarial networks with gradient penalty," in *2021 6th International Conference on Computational Intelligence and Applications (ICCIA)*, Xiamen, China, 2021, pp. 118–122.